

תרגיל בית 3:

שאלה 1:

א.

i. $2^{\log_7 n} = O(n)$. כלומר, על פי הגדרה - $2^{\log_7 n} \leq c * n$ כאשר c הינו קבוע גדול מאפס. מכאן נמשיך בשינויים אלגבריים - $2^{\log_7 n} = 2^{(\log_2 n)/(\log_2 7)} = n^{1/(\log_2 7)} \leq n \leq c * n = O(n)$ עבור כל בסיס לוגריתם גדול שווה משתיים הביטוי היה שווה לחזקה נמוכה מאחד של n ולכן בהכרח קטן מ- n וגם מ- $c * n$ ולכן $2^{\log_7 n} = O(n)$ כדרוש.

ii. $\sum_{a=1}^n (\sum_{b=a}^n (b)) = O(n^2)$. על פי ההגדרה - $\sum_{a=1}^n (\sum_{b=a}^n (b)) \leq c * n^2$. כאשר c קבוע גדול מאפס. מכאן נמשיך אלגברית -

$$\begin{aligned} \leftarrow \sum_{a=1}^n (\sum_{b=a}^n (b)) &\leq (1 + 2 + \dots + n) + (2 + \dots + 3 + \dots + n) + \dots + n \\ &\leq n + n + \dots + n = n * n = n^2 \leq c * n^2 \end{aligned}$$

הביטוי ולכן הביטוי הוא מסדר גודל של $O(n^2)$.

iii. $\log_2((\sum_{k=1}^n (k^{17}))) = O(\log_2 n)$. כלומר, $\log_2((\sum_{k=1}^n (k^{17}))) \leq c * \log_2 n$. והרי כי

$$\begin{aligned} \leftarrow \log_2((\sum_{k=1}^n (k^{17}))) &= 4 * \log_2(\sum_{k=1}^n (k^{17})) = 4 * \log_2(1^{17} + 2^{17} + \dots + n^{17}) \\ &\leq 4 * \log_2(n * n^{17}) = 4 * \log_2(n^{18}) = 4 * 18 * \log_2(n) = 72 * \log_2(n) \leq c * \log_2 n \end{aligned}$$

כדרוש כאשר c קבוע גדול מאפס.

iv. $2^{n^3} \neq O(3^{n^2})$. זה משום שעל פי הגדרה

$$2^{n^3} \leq c * 3^{n^2} \leftarrow 2^{n^3} = O(3^{n^2}) \leftarrow c \geq (2^{n^3})/(3^{n^2})$$

והרי שהצד הימני של האי שוויון שואף מאוד מהר לאינסוף ולכן לא קיים קבוע חיובי c שבאמצעותו ניתן לחסום את הפונקציה 2^{n^3} בעזרת הפונקציה 3^{n^2} . כלומר הטענה הופרכה.

v. אפריך את הטענה על ידי מתן דוגמא נגדית - $f(n) = \log_2 n^8$. ידוע כי $f(n) = O(\log_2 n)$ ונניח בשלילה כי $2^{f(n)} = 2^{\log_2 n^8} = O(n)$. כלומר מתקיים שקיים קבוע c כך ש- $2^{\log_2 n^8} \leq c * n$

$$\leftarrow \log_2 2^{\log_2 n^8} = \log_2 n^8 = 8 * \log_2 n \leq \log_2 (c * n) = \log_2 c + \log_2 n \leftarrow c \geq n^7 \leftarrow \log_2 c \geq \log_2 n^7 \leftarrow \log_2 c \geq 7 * \log_2 n$$

סתירה.

vi. טענה זו נכונה. על מנת שיתקיים כי $2^{f(n)} = O(n)$, אזי $f(n)$ חייבת להיות פונקציה לוגריתמית בבסיס 2 שתוכנה ליניארי מפני שאם לא אז הנעלם ישאר בחזקה והרי שזה ביטוי אקספוננציאלי שלא ניתן לחסום על ידי n וקבוע חיובי. לפי צורך זה, בגלל ש- $f(n)$ פונקציה לוגריתמית בבסיס 2 אזי על פי תוכן הלוגריתם ניתן לחסום את הפונקציה על ידי $\log(n)$ וקבוע c . כלומר, $f(n) = O(\log_2 n)$ כנדרש.

ב.

I. סיבוכיות הפונקציה הינה $O(n^2)$ משום שלולאת הפונקציה מבצעת n איטרציות ובכל איטרציה הפקודה extend מבצעת $2 * i$ פעולות כאשר i רץ מאפס עד $n - 1$. כלומר, באיטרציה הראשונה לא יבוצעו פעולות, בשנייה 2 פעולות ובאיטרציה ה- n יבוצעו $2 * n$ פעולות. נחשב את מספר הפעולות כסכום סדרה חשבונית: $a_1 = 0, a_n = 2 * n, S_n = (n(0 + 2 * n))/2 = n^2 = O(n^2)$.

II.

סיבוכיות הפונקציה הינה $O(n^2)$ משום שלולאת הפונקציה מבצעת n איטרציות ובכל איטרציה פעולת extend מבצעת מספר קבוע של פעולות $2 * n$. כלומר, באיטרציה הראשונה ובאיטרציה ה- n יבוצעו $2 * n$ פעולות אזי סך כל הפעולות הנעשות הינו $2n^2 = O(n^2)$. כלומר ניתן לחסום את פונקצית מספר הפעולות על ידי קבוע חיובי.

III.

סיבוכיות הפונקציה הינה $O(n)$ משום שלולאת הפונקציה מבצעת n איטרציות והרי כי בכל איטרציה מספר הפעולות שנעשות הוא על פי ה-Range והרי כי הטווח שלו קבוע $len(lst) + 500 - (len(lst) - 500) = 1000$. כלומר, על ידי קבוע 1000 ופונקציה ליניארית n ניתן לחסום את פונקציית פעולות הפונרציה $(1000 * n)$.

IV.

סיבוכיות הפונקציה היא $O(3n * 3^n)$ משום שלולאת הפונקציה מבצעת n איטרציות כאשר בכל איטרציה נוצרת רשימה חדש בה הרשימה הישנה ועוד רשימה באורך כפול מאורך הרשימה המקורית. כלומר, באיטרציה הראשונה יבוצעו $3n$ פעולות, בשנייה $9n$ וב- n יבוצעו $3^n * n$ פעולות, ניתן להגיע לכך באמצעות סכום סדרה הנדסית:
 $a_1 = 3n, q = 3, S_n = (3n(3^n - 1))/(3 - 1) = 0.5 * (3n * 3^n - 3n) = O(3n * 3^n)$

ג.

הפונקציה תיכנס ללולאה אינסופית משום שהפונקציה מבצעת איטרציה על כל איבר ברשימה, על כל איבר ברשימה יוסף איבר חדש לרשימה ("a") ולכן היא תתארך באחד. לפיכך, על הלולאה יהיה לבצע איטרציה נוספת. דבר זה יקרה בכל איטרציה של הלולאה ולכן היא לעולם לא תעצור גם בקנה מידה אינסופי.

שאלה 2:

א.

2. סיבוכיות זמן הריצה של הפעולה שכתבתי הינה $O(k)$ כאשר k הוא הערך המינימלי עבורו $f(k) \geq 0$.
 וזה משום שבמקרה ואכן יש לקלט פיתרון, אזי נעשה לולאה שמסתיימת כאשר $f(n) = 0$ באמצעות הצבה של ערכים גדלים ובכל איטרציה של הלולאה נבצע $O(1)$ פעולות. במקרה ואין לפונקציה פיתרון אזי הפונקציה תבצע רק $O(1)$ פעולות ולכן בניתוח מקרה גרוע סיבוכיות זמן הריצה של הפעולה הינו $O(k)$, משום שאנו מבצעים בדיקה של ערכים טבעיים אחד-אחד עד שאנו מגיעים לטבעי עבורו המשוואה גדולה/שווה לאפס ולכן במקרה הגרוע זהו $O(k)$.

ג.

2. הפונקציה שכתבתי עונה על דרישות הסיבוכיות משום שהיא מוצאת את הטווח בו אכן יש פיתרון למשוואה באמצעות התקרבות לשורש בצורה מעריכית. אני כל פעם משנה את הטווח בחזקות של 2 ובשל כך למרות השימוש בלולאה, אני עדיין נשאר בסיבוכיות של $\log$. כאשר אני מוצא את הטווח המתאים אני מפעיל אותו על הפונקציה מהסעיף הקודם שהיא הרי פועלת בסיבוכיות של $O(\log(b - a))$. בטווח שמורץ עליו הפונקציה מהסעיף הקודם קיים השורש של הפונקציה (אם יש לפונקציה שורש) ולכן הפעולה רצה ב- $O(\log(k))$.

שאלה 3:

ב.

סיבוכיות זמן הריצה של הפעולה הינה $O(n * (k + 1))$ משום שבמקרה הגרוע ביותר, n/k יהיה שווה לערך העליון שלו והרי שהלולאת הפעולה מבצעת n/k איטרציות ובכל איטרציה מלבד הפעולות הקבועות שסיבוכיות זמן הריצה שלהן היא $O(1)$ (בדיקות if ו-append) מבצעים $k^2 + k$ selection_sort. וביצירת הרשימה החדשה מבצעים פעולות כאורך הרשימה - k . לפיכך, מספר הפעולות שה"כ הינו $(k^2 + k) * n/k = (k + 1) * n = O(n * (k + 1))$.

ד.

סיבוכיות האלגוריתם הינה חיבור סיבוכיות הפעולה מסעיף א' והפעולה מסעיף ג' שכן האלגוריתם קורא לפעולת המיזוג רשימות עם פעולת סידור הרשימות כקלט. לפיכך, קודם כל מתבצע סידור הרשימות ולאחר מכן מיזוג הרשימות לפי הרשימה בעלת הרשימות המסודרות שחושבה, לכן, מספר הפעולות המתבצעות הינו חיבור מספר הפעולות של שתי הפונקציות. נבחן את המקרה הגרוע ביותר - ידוע כי $k \leq n$ ולכן במידה ו- k בערכו המקסימלי (n) אזי

$$n * (k + 1) + n \log_2(n/k) = n^2 + n + 0 \leq c * n^2 = O(n^2)$$

$$n * (k + 1) + \log_2(n/k) = c * n + n \log_2(n) \leq c * n \log(n) = o(n \log(n))$$

 שתי הפעולות ככל ש- n גדול יותר כך גם הסיבוכיות אך עבור פעולת המיזוג ככל ש- k קטן יותר אז הסיבוכיות גדולה יותר ואילו עבור הסידור ככל ש- k קטן גדול יותר אז הסיבוכיות גדולה יותר ולכן בניתוח מקרה גרוע מהמשוואות שהצגתי, סיבוכיות זמן הריצה היא מסדר גודל של $n \log_2 n$ של $n * (k + 1) + n \log_2 n \leq c * n(\log_2(n/k) + k) = O(n(\log(n/k) + k))$ (מקרי הקצה של k לא דרושים לסעיף זה).

ה.

כפי שהצגתי בסעיף הקודם, על מנת שסיבוכיות זמן הריצה של הפונקציה מסעיף די (sort_by_block_merge) תהיה זהה לזו של merge_sort, כלומר, $o(n \log(n))$ צריך שיתקיים כי $k = 1/n$ וזה משום שאז $n * (k + 1) + \log_2(n/k) = n + 1 + n \log_2(n/(1/n)) = n + 1 + 2n \log_2(n) \leq c * n \log(n) = o(n \log(n))$ כלומר, קיבלנו שכאשר $k = 1/n$ סיבוכיות הפונקציה זהה לשל merge_sort - $o(n \log(n))$ כדרוש.

שאלה 4:

א.

הערך השלם הקטן ביותר של n כך שזמן הריצה שלו הוא $300n^2$ רץ מהר יותר מאלגוריתם שזמן ריצות הוא 2^n הוא $n = 17$. מצורף הקוד עימו הגעתי לתוצאה זו:

```
>>> f = lambda x: 300*(x**2)
>>> g = lambda x: 2**x
>>> n = 1
>>> while f(n) >= g(n):
>>>     n +=1

>>> print(n)
17
```

ב.

$f(n)$	שנייה	דקה	שעה	יום	חודש	שנה	100 שנים
$\log(n)$	$2^{1 \cdot 10^6}$	$2^{6 \cdot 10^7}$	$2^{3.6 \cdot 10^9}$	$2^{8.64 \cdot 10^{10}}$	$2^{2.592 \cdot 10^{12}}$	$2^{31104 \cdot 10^9}$	$2^{31104 \cdot 10^{11}}$
$n^{(0.5)}$	$1 * (10^{12})$	$3.6 * (10^{15})$	$1.296 * (10^{19})$	$7.46496 * (10^{21})$	$6.718464 * (10^{24})$	$9.6745882 * (10^{26})$	$9.6745882 * (10^{30})$
n	1000000	60000000	$3.6 * (10^9)$	$8.64 * (10^{10})$	$2.592 * (10^{12})$	$31104 * (10^9)$	$31104 * (10^{11})$
$n \cdot \log(n)$	62746	2801417	133378058	2755147513	$7.18709 * 10^{10}$	$7.8709 * 10^{11}$	$6.76995 * 10^{13}$
n^2	1000	7745	60000	293938	1609968	5577096	55770960
n^3	100	391	1532	4420	13736	31448	14597
2^n	19	25	31	36	41	44	51
$n!$	9	11	12	13	15	16	17

ג.

- **חיפוש בינארי** - ניתן לממש חיפוש בינארי בסיבוכיות מקום של $O(1)$, כלומר, המקום בזיכרון שנלקח על ידי האלגוריתם הוא לכל אורך רשימה שתתקבל כקלט.
- **מיון בחירה** - ניתן לממש מיון בחירה בסיבוכיות מקום של $O(1)$. כלומר, המקום בזיכרון שנלקח על ידי האלגוריתם הוא לכל אורך רשימה שתתקבל כקלט.
- **מיזוג** - סיבוכיות מקום של $O(n + m)$ משום שבאלגוריתם זה אנו יוצרים רשימה חדשה באורך שתי הרשימות (יחדיו) כאשר n מייצג את אורך רשימה a ואילו m מייצג את אורך רשימה b .

שאלה 5:

ב.

סיבוכיות זמן הריצה של הפונקציה מסעיף אי כוללת ב- n ו/או k הינה מסדר גודל של $O(n * k^3)$. זוהי סיבוכיות זמן הריצה משום שבפעולתנו, אנו מממשים שלוש לולאות מקוננות ככה שעבור כל ערך מ- $0 \dots k$ עוברים על כל ערך מ- $0 \dots k$ ועוברים עוברים על כל ערך מ- $0 \dots k$. וזה על מנת לגשת לכל תא שברשימה התלת מימדית שיצרנו כעזר. יתרה מכך, בכל איטרציה של הלולאות האלה, אנו בודקים האם קיים ערך בתוך הרשימה שזה נעשה בסדר גודל של $O(n)$. כמו כן, בעת יצירת רשימת העזר של הטאפלים (כנדרש בשאלה), יש את אותה מערכת לולאות (אם כי נעשה ב- list comprehension) ולכן מספר הפעולות המתבצעות בריצה הוא $k^3 + n * k^3 \leq 2 * n * k^3 = O(n * k^3)$ כנדרש.

ד.

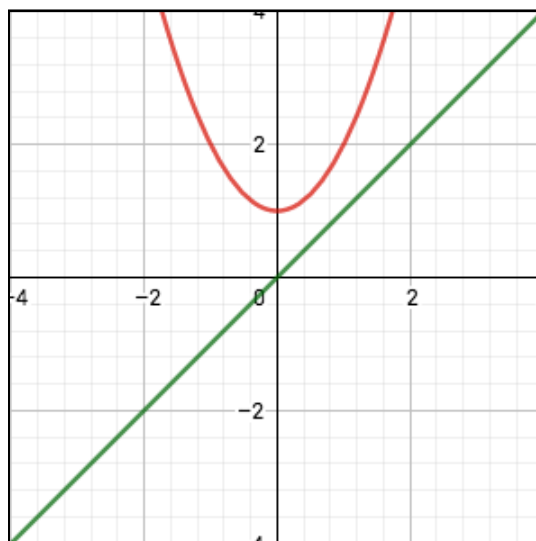
סיבוכיות זמן הריצה של הפונקציה שכתבתי בסעיף הקודם כוללת ב- n ו/או k הינה מסדר גודל של $O(n^2)$. וזה משום שבכל ריצה של הפעולה אנחנו ראשית מעתיקים את הרשימה שניתנה לרשימה שתוחזר והרי שהפעולה של פייתון copy פועלת בסדר גודל של $O(n)$. יתרה מכך, לאחר מכן אנו מממשים את אלגוריתם selection sort ומתאימים אותו לשאלה ומבצעים איטרציה על כל איבר ברשימה ובכל איטרציה שכזו מבצעים איטרציה על שאר האיברים. כלומר, בניתוח מקרה גרוע הוא יפעל בסיבוכיות של $O(n^2)$ ולכן $n^2 + n \leq 2n^2 = O(n^2)$ וזוהי סיבוכיות הפונקציה שפעמים רבות איטית יותר מאשר הפונקציה מסעיף אי שכן אורך הרשימה פעמים רבות גדול יותר מטווח המספרים שבטאפלים שבו, אך ניתן לראות כי פעולות הפונקציה כלל אינן תלויות בטווח תוכן הטאפלים (k).

שאלה 6:

ב.

עבור המקרה הראשון המצורף למטה, הפעולה equal תחזיר None ותודפס השורה מתוך הפעולה NR שמתחילה ב- "No Convergence". זה משום שאין לפונקציות $f(x) = x^2 + 1$ ו- $g(x) = x$ נקודת חיתוך, נבדוק זאת (נשווה את הפונקציות) ונקבל $x^2 - x + 1 = 0$ ולמשוואה זו אין פיתרון. כלומר, אין לפונקציות שהתקבלו כקלט נקודת חיתוך ולכן הפלט המתאים של ניוטון רפסון (אין נקודת אפס למשוואת ההפרש $(x^2 - x + 1 = 0)$ יתקבל. בצורה גרפית נראה זאת כך:

```
equal(lambda x:x, lambda x:x**2+1)
```



- עבור המקרה השני אשר מצורף מטה, הפעולה equal תחזיר None ותודפס השורה מתוך NR שמתחילה ב- "Zero derivative". זה משום שכאשר נחפש עבור הפונקציות $f(x) = x$ ו- $g(x) = x + 1$ נקודות חיתוך ונמצא את הפונקציה המתארת זאת (שמועברת ל-NR, כלומר, פונקציית ההפרשים) על ידי השוואת הפונקציות ונקבל $1 = 0$, כלומר, אין נקודת חיתוך (פלט "None") ויתרה מכך, פונקציית ההפרשים (המועברת ל-NR) היא $h(x) = x + 1 - x = 1$ שהיא פונקציה המקבילה לציר איקס, כלומר, אין שינוי בערכי ה- y של הפונקציה ולכן נגזרתה אפס ויודפס "Zero derivative". הדבר מוצג גרפית מטה:

```
equal(lambda x:x, lambda x:x+1)
```

	$f(x) = x$
	$g(x) = x + 1$
	$h(x) = 1$

